

Experiment-2 Report

Fixed-Point Arithmetic Operations using Verilog and MATLAB Verification

Aslin Garvasis
EE24BTECH11008

1 Aim

To perform basic fixed-point arithmetic operations (addition, subtraction and multiplication) on an array of fixed-point data using Verilog, write the computed results into text files, and verify the correctness of Verilog outputs using MATLAB reference computations. The operands are taken from the previous experiment in the formats Q(3,14) and Q(5,12).

2 Given Specifications

- Operand-1 format: Q(3,14)
- Operand-2 format: Q(5,12)
- Operations to be performed:
 - Addition
 - Subtraction
 - Multiplication
- Verification tool: MATLAB reference program
- Output requirement: Write results to files and verify MATLAB vs Verilog plots

3 Theory of Fixed-Point Representation

Fixed-point numbers are represented in the form Q(m,n), where:

- m = number of integer bits (excluding sign bit)
- n = number of fractional bits

A fixed-point number is stored as an integer X_{int} and its real value is obtained as:

$$X = \frac{X_{int}}{2^n} \quad (1)$$

3.1 Operand Formats Used

- Q(3,14): Fractional bits = 14, scaling factor = 2^{14}
- Q(5,12): Fractional bits = 12, scaling factor = 2^{12}

4 Requirement of Q-Format Conversion

For addition and subtraction, both operands must have the same number of fractional bits. Since the given operands have different fractional resolutions, a conversion is required.

4.1 Conversion of Operand-2 from Q(5,12) to Q(5,14)

Operand-2 has 12 fractional bits, but operand-1 has 14 fractional bits. Therefore operand-2 must be converted to 14 fractional bits by left shifting by $(14 - 12) = 2$ bits:

$$X_2(Q14) = X_2(Q12) \times 2^{(14-12)} = X_2(Q12) \times 4 \quad (2)$$

This conversion ensures that both operands have the same fractional resolution before performing addition or subtraction.

5 Fixed-Point Arithmetic Operations

5.1 Addition

After converting operand-2 to Q14:

$$Y_{add} = X_1(Q14) + X_2(Q14) \quad (3)$$

The output remains in Q14 format.

5.2 Subtraction

After converting operand-2 to Q14:

$$Y_{sub} = X_1(Q14) - X_2(Q14) \quad (4)$$

The output remains in Q14 format.

5.3 Multiplication

Multiplication increases the number of fractional bits. Since:

- Operand-1 is Q(3,14)
- Operand-2 is Q(5,12)

The raw multiplication result has:

$$n_{out} = 14 + 12 = 26 \quad (5)$$

Thus the raw multiplication output is in Q(?,26):

$$Y_{mul,raw} = X_1 \times X_2 \quad (6)$$

To convert the multiplication output back to Q14, the result must be right shifted by $(26 - 14) = 12$ bits:

$$Y_{mul}(Q14) = \frac{Y_{mul,raw}}{2^{12}} \quad (7)$$

Hence, the final multiplication output is also represented in Q14 format.

6 Methodology

6.1 MATLAB Operand File Generation

MATLAB was used to generate and store the fixed-point integer arrays corresponding to:

- Operand-1 in Q(3,14)
- Operand-2 in Q(5,12)

The values were saved into text files as signed decimal integers (one sample per line). These files were later used as inputs for the Verilog simulation.

6.2 Verilog Implementation

A Verilog module was designed to perform the fixed-point operations. A separate test-bench was used to:

- Read operand samples from input files
- Apply inputs to the design module
- Perform addition, subtraction and multiplication
- Write outputs to separate text files

The Verilog outputs were written into:

- verilog_add.txt
- verilog_sub.txt
- verilog_mul.txt

All values were written as signed decimal integers in Q14 format.

6.3 Waveform Generation using GTKWave

During Verilog simulation, a waveform dump file (`waveform.vcd`) was generated using `$dumpfile` and `$dumpvars`. The waveform was viewed using GTKWave to observe the signals and verify the fixed-point computations at the bit level.

The following signals were observed in GTKWave:

- Q(5,12) operand (Input operand-2)
- Q(5,14) converted operand (operand-2 shifted left by 2 bits)
- Addition output (Q14)
- Subtraction output (Q14)
- Multiplication output (Q14)

GTKWave display format was set to **binary** to visualize the values in 2's complement form.

6.4 MATLAB Verification

MATLAB was used to verify Verilog outputs by:

- Reading Verilog output files
- Performing the same operations in MATLAB using integer arithmetic
- Comparing MATLAB results with Verilog results
- Plotting MATLAB and Verilog outputs

If the Verilog implementation is correct, MATLAB and Verilog output plots should overlap.

7 Results

7.1 GTKWave Waveform Screenshot

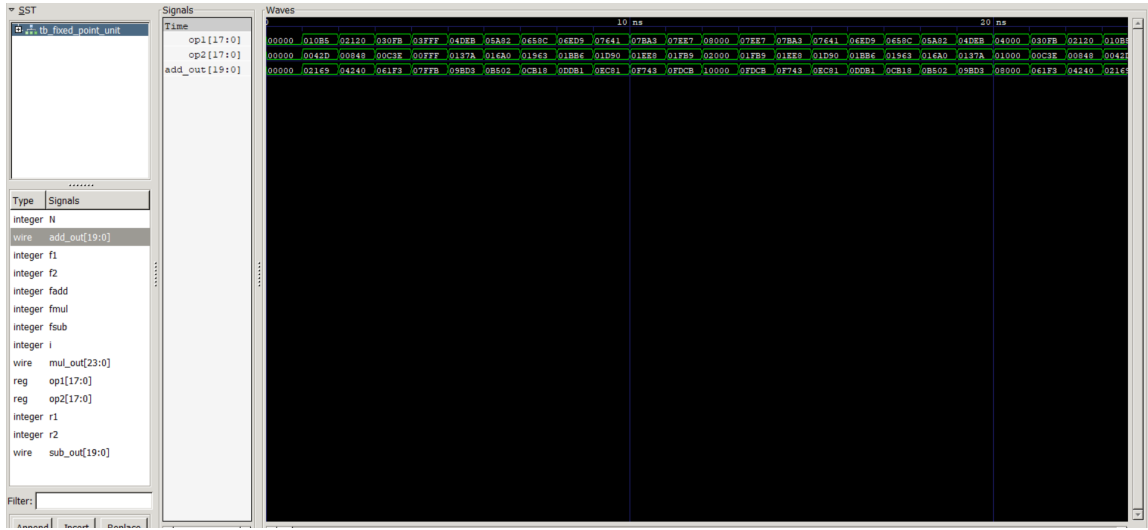


Figure 1: GTKWave waveform showing Q12, Q14, Add, Sub and Mul outputs in binary format

7.2 Plots for Addition Output

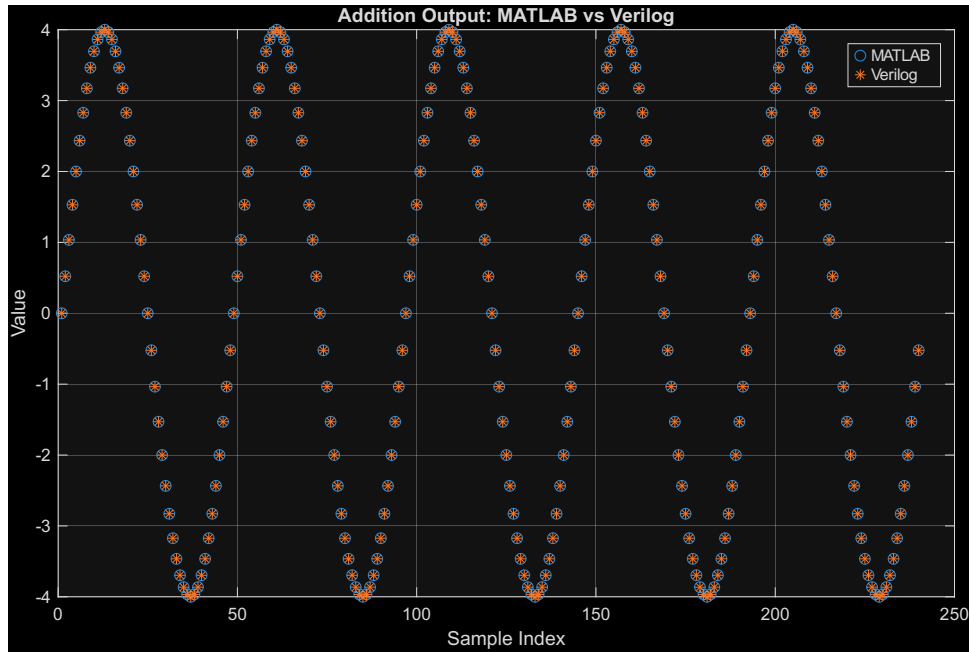


Figure 2: MATLAB vs Verilog Addition Output (Q14)

7.3 Plots for Subtraction Output

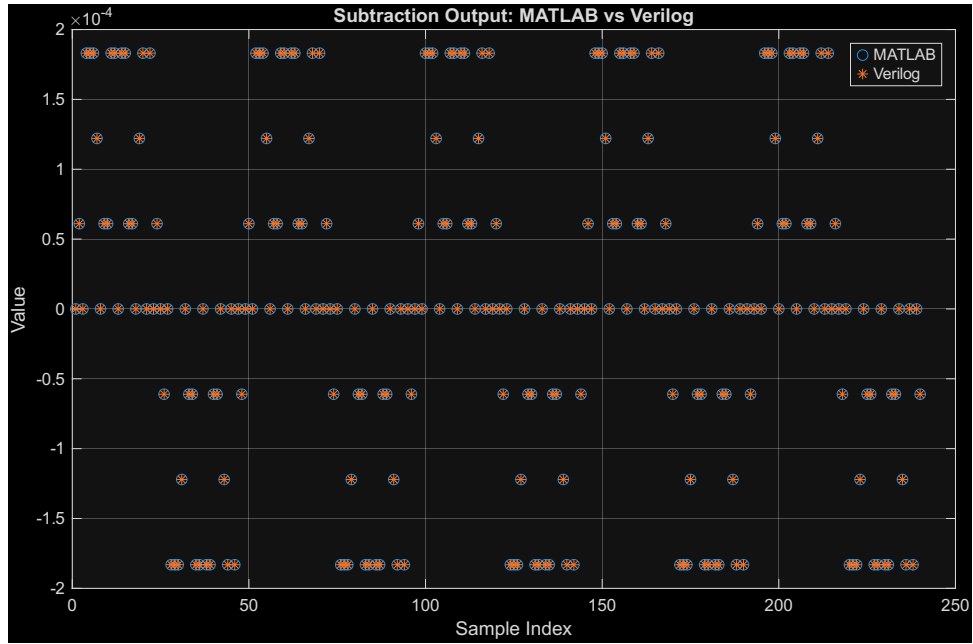


Figure 3: MATLAB vs Verilog Subtraction Output (Q14)

7.4 Plots for Multiplication Output

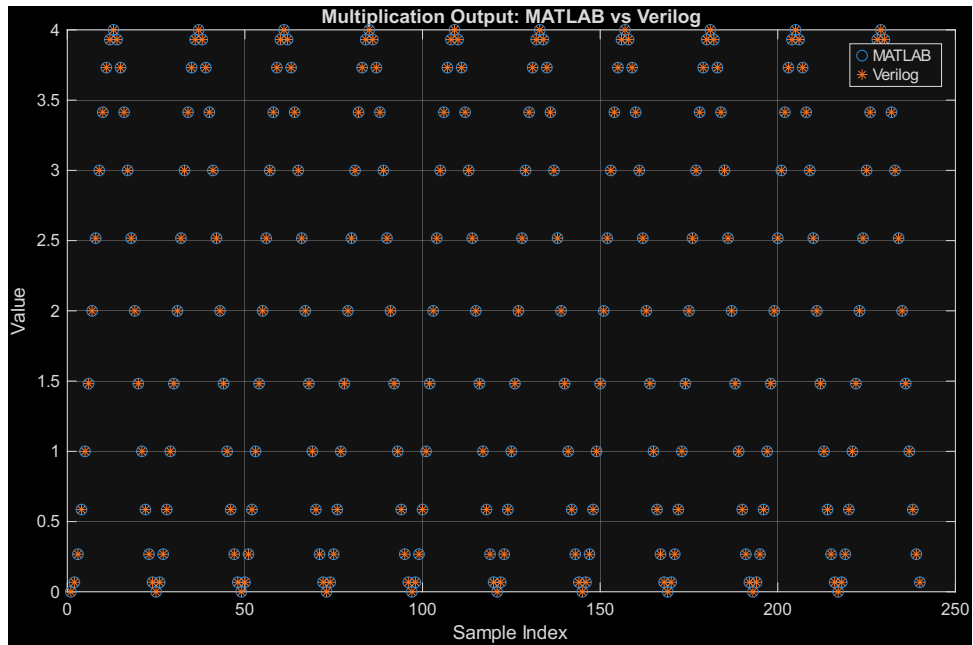


Figure 4: MATLAB vs Verilog Multiplication Output (Q14)

8 Comparison Table

A comparison table was generated in MATLAB for the first few samples of each operation to verify that MATLAB and Verilog outputs match.

Index	MATLAB_ADD	Verilog_ADD	MATLAB_SUB	Verilog_SUB	MATLAB_MUL	Verilog_MUL
1	0	0	0	0	0	0
2	0.52203	0.52203	6.1035e-05	6.1035e-05	0.068115	0.068115
3	1.0352	1.0352	0	0	0.26788	0.26788
4	1.5305	1.5305	0.00018311	0.00018311	0.58557	0.58557
5	1.9997	1.9997	0.00018311	0.00018311	0.99969	0.99969
6	2.4348	2.4348	0.00018311	0.00018311	1.482	1.482
7	2.8282	2.8282	0.00012207	0.00012207	1.9997	1.9997
8	3.1733	3.1733	0	0	2.5175	2.5175
9	3.4639	3.4639	6.1035e-05	6.1035e-05	2.9997	2.9997
10	3.6954	3.6954	6.1035e-05	6.1035e-05	3.4139	3.4139
11	3.8635	3.8635	0.00018311	0.00018311	3.7316	3.7316
12	3.9655	3.9655	0.00018311	0.00018311	3.9313	3.9313
13	4	4	0	0	4	4
14	3.9655	3.9655	0.00018311	0.00018311	3.9313	3.9313
15	3.8635	3.8635	0.00018311	0.00018311	3.7316	3.7316
16	3.6954	3.6954	6.1035e-05	6.1035e-05	3.4139	3.4139
17	3.4639	3.4639	6.1035e-05	6.1035e-05	2.9997	2.9997
18	3.1733	3.1733	0	0	2.5175	2.5175
19	2.8282	2.8282	0.00012207	0.00012207	1.9997	1.9997
20	2.4348	2.4348	0.00018311	0.00018311	1.482	1.482
21	2	2	0	0	1	1
22	1.5305	1.5305	0.00018311	0.00018311	0.58557	0.58557
23	1.0352	1.0352	0	0	0.26788	0.26788
24	0.52203	0.52203	6.1035e-05	6.1035e-05	0.068115	0.068115
25	0	0	0	0	0	0
26	-0.52203	-0.52203	-6.1035e-05	-6.1035e-05	0.068115	0.068115
27	-1.0352	-1.0352	0	0	0.26788	0.26788

Figure 5: MATLAB vs comparison table

9 Discussion

- Addition and subtraction required Q-format alignment since operands had different fractional bits.
- Operand-2 in Q(5,12) was shifted left by 2 bits to convert it into Q(5,14).
- Multiplication resulted in Q26 output which was shifted right by 12 bits to obtain Q14 output.
- GTKWave waveform confirmed the operations at the binary level.
- MATLAB plots matched Verilog outputs, confirming correctness.

10 Conclusion

Fixed-point addition, subtraction, and multiplication were successfully implemented in Verilog using operand arrays stored in Q(3,14) and Q(5,12) formats. Proper Q-format conversion was performed before addition/subtraction and after multiplication. The generated Verilog outputs were written into files and verified using MATLAB reference computations. GTKWave waveform visualization confirmed the correctness of binary fixed-point computations. MATLAB verification plots confirmed that Verilog and MATLAB results were identical.